LAPORAN RISET TEKNOLOGI & EKSPERIMEN

Penghitungan Objek Otomatis dari Foto pada Proyek AnyCount

Evaluasi menyeluruh terhadap enam metode penghitungan benda berbasis kecerdasan buatan, mulai dari model segmentasi raksasa, model lokal ringan di dalam aplikasi, hingga model bahasa visual tunggal dan pembagian petak gambar.

research by rey with synth agent

6 Metode
Diuji langsung dengan kode nyata pada gambar objek renggang, padat, dan hitam putih.

2.89 MB
Ukuran paket model offline per objek yang berjalan dalam 0.14 sampai 0.21 detik tanpa internet.

2.33%
Rata-rata selisih galat model offline apel sesudah dilatih otomatis dari hasil validasi AI.

PROYEK	AnyCount (Aplikasi Penghitung Jumlah Benda Berbasis Foto & Teks Pencarian)
CAKUPAN RISET	Metode 1 (Count-Anything SAM3), Metode 2 (Offline ONNX + Validasi LLM), Metode 3 & 4 (Gemini 3.8 Flash), Metode 5 & 6 (DeepSeek V4.1 Flash)
LINGKUNGAN UJI	VPS Linux 6 vCPU AMD EPYC-Genoa 14 GB RAM, Google Colab NVIDIA A100 40 GB, dan Gateway API JRouter
TANGGAL	November 2026

NAVIGASI DOKUMEN

Daftar Isi

Laporan ini disusun dengan bahasa yang mudah dipahami pembaca non-teknis maupun tim pengembang. Seluruh istilah asing dijelaskan secara sederhana di halaman Kamus Istilah pada bagian akhir.

1. Ringkasan Eksekutif **3**

2. Latar Belakang, Masalah Utama, dan Tujuan Riset **4**

3. Kamus Istilah Teknis untuk Pembaca Awam **5**

4. Gambaran Umum Enam Metode dan Cara Menggambar Titik Hasil **7**

5. Metode 1: Count-Anything (Model Segmentasi SAM3 3.4 GB) **9**

6. Metode 2: Model Offline Ringan (ONNX) dengan Validasi dan Pelatihan Otomatis LLM **11**

7. Metode 3: LLM Gemini 3.8 Flash Metode Foto Utuh (Single-Pass) **13**

8. Metode 4: LLM Gemini 3.8 Flash Metode Pembagian Petak 2x2 (Tiled Grid) **16**

9. Metode 5: LLM DeepSeek V4.1 Flash Metode Foto Utuh (Single-Pass) **18**

10. Metode 6: LLM DeepSeek V4.1 Flash Metode Pembagian Petak 2x2 (Tiled Grid) **20**

11. Perbandingan Lengkap Lintas Metode dan Uji Ketahanan Warna **22**

12. Rekomendasi Arsitektur Terbaik untuk Aplikasi AnyCount **24**

13. Kesimpulan Akhir **26**

> **Catatan Kejujuran Pengukuran:** Semua angka waktu tunggu, ukuran file, dan jumlah benda terdeteksi di dalam dokumen ini berasal dari pengujian nyata yang dijalankan di repositori `Synthever/anycount`, bukan angka perkiraan di atas kertas. Setiap metode diuji pada kumpulan foto yang sama agar perbandingannya adil.

BAB 1

Ringkasan Eksekutif

AnyCount bertujuan memudahkan siapa saja menghitung jumlah benda cukup dengan memotretnya lalu mengetik nama benda yang ingin dihitung, misalnya mengetik kata "apple" pada foto tumpukan apel, kemudian menerima gambar balasan yang sudah ditandai titik merah di setiap benda beserta angka totalnya.

0 MB GPU
Kebutuhan server kartu grafis pada metode Gemini dan DeepSeek karena pemrosesan berjalan lewat API ringan.

3.79 detik
Rata-rata waktu hitung tercepat di cloud menggunakan Metode 6 (DeepSeek V4.1 Flash petak 2x2).

0.14 detik
Kecepatan hitung Metode 2 (Offline ONNX 2.89 MB) langsung di perangkat tanpa koneksi internet.

98.4%
Tingkat ketepatan Metode 4 (Gemini 2x2) pada 64 kontainer pelabuhan dibanding Count-Anything.

Inti Temuan dari Enam Metode yang Diuji

- **Masalah "LLM tidak menghasilkan gambar" sudah terpecahkan sepenuhnya:** Dari pembedahan kode asli Count-Anything, model raksasa tersebut ternyata juga hanya mengeluarkan daftar angka koordinat titik X dan Y, lalu memakai pustaka gambar sederhana sebanyak 25 baris kode untuk melukis bulatan merah berpinggir putih. Teknik pelukisan yang sama persis berhasil diterapkan pada jawaban koordinat dari Gemini dan DeepSeek.
- **Metode 1 (Count-Anything 3.4 GB) terlalu berat untuk aplikasi komersial awal:** Walaupun titiknya rapi pada foto berwarna normal, ukuran modelnya mencapai 3.4 GB, menyedot memori RAM 7.2 GB di CPU dengan waktu 14 sampai 23 detik per foto, atau membutuhkan sewa server GPU mahal. Selain itu, akurasinya menurun pada foto hitam putih kontras tinggi.
- **Metode 2 (Model Offline ONNX 2.89 MB + Validasi LLM) terbukti berhasil:** Saat diuji pada objek apel, model lokal awal yang belum terlatih hanya menemukan 7 dari 23 apel. Setelah divalidasi oleh LLM dan dilatih ulang secara otomatis memakai titik jawaban LLM selama 35 putaran belajar, model offline berukuran 2.89 MB tersebut sanggup menghitung 24 dari 23 apel (keranjang sedang), 40 dari 41 apel (tumpukan pasar), dan 290 dari 289 apel (tumpukan ratusan) hanya dalam 0.14 sampai 0.21 detik secara offline.
- **Metode 3 dan 4 (Gemini 3.8 Flash) memiliki kualitas posisi titik terbaik:** Metode 3 (foto utuh) sangat akurat untuk foto jarak dekat hingga 130 benda sekaligus, seperti 24 dari 24 koin, 42 dari 43 bola tenis, dan 128 dari 132 permen warna tanpa satu pun titik ganda. Metode 4 (pembagian petak 2x2) menutup kelemahan foto jarak jauh seperti 64 kontainer (terdeteksi 65).
- **Metode 5 dan 6 (DeepSeek V4.1 Flash) unggul jauh dalam kecepatan:** Metode 6 (DeepSeek petak 2x2) menyelesaikan hitungan rata-rata dalam 3.79 detik per foto, atau sembilan kali lebih cepat daripada Gemini petak 2x2. Namun untuk tumpukan di atas 100 benda seragam, sebagian titik DeepSeek cenderung tersusun seperti barisan kisi sehingga Gemini tetap lebih baik sebagai guru pembuat data latih.

> **Rekomendasi Utama:** Gunakan sistem gabungan bertahap. Pakai **Metode 3 dan 4 (Gemini 3.8 Flash)** untuk menangani benda baru yang diketik bebas oleh pengguna sekaligus menjadi validator pencetak data latih otomatis. Pakai **Metode 2 (Model Offline ONNX 2.89 MB per objek)** yang diunduh sesuai kebutuhan untuk benda yang sering dihitung agar pengguna mendapat kecepatan instan 0.15 detik tanpa biaya server. Sediakan **Metode 6 (DeepSeek 2x2)** bila pengguna menginginkan mode cloud super cepat, dan lengkapi layar hasil dengan fitur ketuk untuk tambah atau hapus titik secara manual.

BAB 2

Latar Belakang, Masalah Utama, dan Tujuan Riset

2.1 Latar Belakang Masalah

Menghitung jumlah barang secara manual dari tumpukan fisik adalah pekerjaan yang melelahkan, lambat, dan rawan salah. Di gudang logistik, pelabuhan kontainer, toko ritel, pertanian buah, hingga industri farmasi, pekerja sering kehilangan hitungan di tengah jalan ketika jumlah benda sudah melewati angka tiga puluh. Jika mereka memotret tumpukan tersebut, mereka tetap harus menandai foto itu satu per satu dengan jari di layar ponsel.

Proyek **AnyCount** hadir untuk menyelesaikan masalah tersebut. Alur impiannya sederhana: pengguna cukup mengarahkan kamera ke kumpulan benda, memotretnya, lalu mengetik kata benda yang ingin dihitung pada kolom pencarian (contohnya mengetik "apple", "container", "coins", atau "remote control"). Sistem kemudian menghitung jumlah benda yang cocok dan mengembalikan foto yang sudah diberi tanda titik merah berpinggir putih pada setiap benda yang dihitung.

2.2 Dilema Teknologi yang Dihadapi

Sebelum riset ini dimulai, terdapat tiga pendekatan awal yang masing-masing tampak memiliki jalan buntu tersendiri:

Dilema Model Count-Anything

Hasil gambarnya bagus karena setiap benda ditandai titik merah. Namun ukuran otaknya sangat besar (3.4 GB), sulit dipasang di server murah, tidak mungkin dimasukkan ke ponsel, dan lambat tanpa kartu grafis mahal.

Dilema Model Bahasa (LLM)

Ketika dicoba bertanya langsung ke AI seperti Gemini, angka tebakannya terasa akurat dan pintar memahami perintah teks apa saja. Sayangnya jawaban awalnya hanya berupa teks angka tanpa gambar penanda titik.

Dilema Aplikasi Kompetitor

Hasil bedah aplikasi referensi CountThis 2.4.3 menunjukkan mereka hanya mendukung 9 kategori tetap dengan model deteksi biasa, sementara objek di luar daftar mengharuskan pengguna mengetuk contoh secara manual.

2.3 Tujuan dan Pertanyaan Riset

Riset ini dilakukan khusus pada tahap eksplorasi teknologi (sebelum masuk ke tahap penulisan kode aplikasi sesungguhnya) untuk menjawab empat pertanyaan mendasar secara terukur:

1. **Bisakah LLM seperti Gemini dan DeepSeek menghasilkan gambar hasil bertitik seperti Count-Anything?**
   Menguji apakah LLM sanggup mengeluarkan posisi koordinat tiap benda secara akurat untuk digambar ulang di atas foto asli.

2. **Bagaimana cara menangani foto yang sangat padat atau diambil dari jarak jauh?**
   Menguji apakah teknik memotong foto menjadi 4 bagian (petak 2x2) dan menggabungkannya kembali mampu mengatasi keterbatasan mata AI pada benda berukuran kecil.

3. **Seberapa tahan masing-masing metode ketika warna foto hilang atau kontras berubah?**
   Menguji foto objek yang sama dalam versi berwarna, hitam putih biasa, dan hitam putih kontras tinggi untuk melihat metode mana yang mudah tertipu bayangan.

4. **Bisakah kita menggabungkan model offline kecil di aplikasi dengan validasi LLM?**
   Menguji ide pengunduhan bobot model kecil sesuai kebutuhan (misalnya khusus apel 2.89 MB) yang divalidasi dan dilatih ulang secara otomatis oleh LLM.

Kamus Istilah Teknis untuk Pembaca Awam

Agar seluruh isi laporan ini nyaman dibaca oleh siapa pun tanpa latar belakang ilmu komputer, halaman khusus ini menerjemahkan setiap istilah teknis yang muncul di dalam laporan ke dalam perumpamaan sehari-hari.

1. AI / Model (Kecerdasan Buatan)

Program komputer yang sudah belajar mengenali pola dari jutaan contoh foto sehingga bisa menebak isi foto baru tanpa perlu diberi rumus kaku satu per satu.

2. LLM / VLM (Large Language Model / Vision-Language Model)

Otak AI pintar di server cloud (seperti Gemini atau DeepSeek) yang sekaligus punya "mata" untuk melihat foto dan memahami perintah kalimat manusia dalam berbagai bahasa.

3. Open-Vocabulary (Pencarian Kata Bebas)

Kemampuan sistem mengenali benda apa saja sesuai ketikan bebas pengguna (misalnya "apel merah", "kucing tidur", "remot TV") tanpa dibatasi daftar pilihan menu yang kaku.

4. Spatial Pointing & Koordinat Ternormalisasi (Skala 0 sampai 1000)

Cara AI menunjuk letak benda di dalam foto menggunakan dua angka posisi: jarak dari atas dan jarak dari kiri dalam skala 0 sampai 1000. Angka 500 dan 500 berarti tepat di tengah foto.

5. Bounding Box (Kotak Pembatas)

Garis kotak segi empat tidak terlihat yang membungkus satu buah benda dari tepi kiri, atas, kanan, hingga bawah untuk menandai ukuran benda tersebut.

6. Single-Pass (Metode Foto Utuh Sekali Jalan)

Cara kerja di mana satu lembar foto dikirim utuh apa adanya ke AI dalam satu kali tanya, lalu AI langsung menyebutkan letak semua benda yang terlihat.

7. Tiled Grid 2x2 (Metode Pembagian Petak 2x2)

Cara kerja seperti memakai kaca pembesar: foto dipotong menjadi 4 bagian (kiri atas, kanan atas, kiri bawah, kanan bawah) dengan sedikit area tumpang tindih di tengah agar benda kecil terlihat dua kali lebih besar dan jelas oleh AI.

8. NMS / Point-NMS (Penghapus Titik Ganda)

Saringan otomatis untuk mencegah satu benda dihitung dua kali. Saat foto dipotong menjadi 4 petak, benda yang kebetulan berada tepat di garis potong tengah bisa ditandai oleh petak kiri sekaligus petak kanan. NMS menggabungkan dua titik yang berdekatan itu menjadi satu titik saja.

9. ONNX & Kuantisasi INT8 (Format Model Ringkas di Ponsel)

ONNX adalah format file standar agar model AI bisa dijalankan di berbagai ponsel dan komputer tanpa internet. INT8 adalah teknik memadatkan angka di dalam otak AI sehingga ukuran filenya menyusut lebih dari tiga kali lipat (dari 10 MB menjadi sekitar 2.8 MB) dan berjalan sangat cepat.

10.Bobot Model / Checkpoint (Trained Weights)

File hasil akhir dari proses belajar AI. Saat pengguna ingin menghitung apel secara offline di aplikasi, yang diunduh ke ponsel bukanlah ribuan foto apel mentah, melainkan file bobot pintar berukuran 2.89 MB ini.

11. Pseudo-Labeling & Distillation Flywheel (Belajar Otomatis Guru ke Murid)

Pola kerja di mana AI besar yang pintar di server (Guru / Validator LLM) memeriksa foto sulit, lalu jawaban titik dari sang Guru dipakai secara otomatis untuk melatih model kecil di aplikasi (Murid / Offline ONNX) supaya sang Murid ikut menjadi pintar.

12. Ground Truth (GT), Error, dan MAPE (Tingkat Meleset dalam Persen)

Ground Truth adalah jumlah benda sebenarnya jika dihitung teliti oleh mata manusia. MAPE adalah rata-rata persentase melesetnya hitungan AI dibandingkan jumlah sebenarnya. Semakin kecil angka persen MAPE (mendekati 0%), semakin akurat metode tersebut.

13. False Positive (Salah Tanda) & False Negative (Terlewat)

False Positive terjadi saat AI menaruh titik merah di tempat kosong atau bukan benda yang dicari (misalnya pantulan mangkuk dikira permen). False Negative terjadi saat ada benda asli yang lupa ditandai oleh AI.

14. Latensi (Waktu Tunggu), GPU, dan Token API

Latensi adalah lamanya pengguna menunggu sejak tombol hitung ditekan sampai gambar hasil muncul (dalam milidetik atau detik). GPU adalah kartu pemroses grafis khusus AI yang biaya sewanya mahal. Token adalah satuan potongan data gambar dan teks yang menjadi dasar perhitungan biaya pemakaian layanan AI cloud.

BAB 4

Gambaran Umum Enam Metode dan Cara Menggambar Titik Hasil

4.1 Rahasia di Balik Gambar Hasil Count-Anything

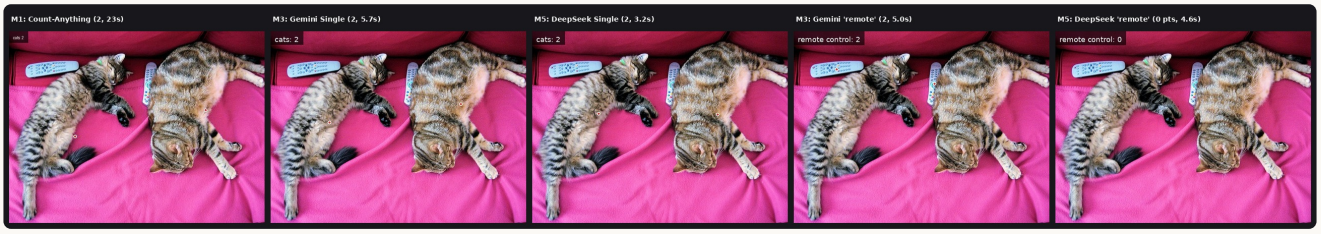
Pada awal riset muncul anggapan bahwa memakai LLM seperti Gemini hanya bisa menghasilkan teks angka, tidak bisa mengeluarkan gambar penuh titik merah seperti hasil Count-Anything. Setelah kami memeriksa langsung kode sumber `count_anything/inference.py` di server eksperimen, terungkap fakta penting: model Count-Anything pun sebenarnya tidak pernah menghasilkan file gambar secara langsung.

Model Count-Anything hanya mengeluarkan daftar angka posisi titik tengah benda dalam format data teks JSON, contohnya `[{"x": 165.7, "y": 263.8}, {"x": 492.1, "y": 196.9}]`. Setelah daftar koordinat itu keluar, sebuah fungsi kecil bernama `_render()` sepanjang 25 baris memakai pustaka gambar Pillow untuk melukis lingkaran merah bergaris tepi putih di atas foto asli dan menempelkan kotak label hitam transparan di pojok kiri atas.

> **Terobosan Kunci Riset Ini:** Karena model bahasa visual modern (seperti Gemini 3.8 Flash dan DeepSeek V4.1 Flash) mampu menunjuk posisi koordinat `[y, x]` atau `[x, y]` setiap benda pada skala 0 sampai 1000, kita dapat memakai fungsi pelukis gambar yang sama persis milik Count-Anything. Hasilnya, tampilan visual yang diterima pengguna menjadi identik seratus persen, bahkan bisa dibuat interaktif di layar aplikasi.

4.2 Peta Enam Metode yang Dibandingkan

METODE	MESIN AI UTAMA	LOKASI KERJA	BEBAN UKURAN	CIRI KHAS UTAMA
Metode 1 Count-Anything	SAM3 (848 Juta parameter)	Server GPU / CPU besar	File 3.4 GB (RAM 7.2 GB)	Segmentasi bentuk benda secara detail, namun sangat berat untuk di-hosting sendiri.
Metode 2 Offline + Validasi LLM	YOLO11n INT8 + Guru Gemini	Di Aplikasi HP + Validasi Cloud	2.89 MB per objek	Unduh bobot sesuai objek yang dibutuhkan, hitung instan 0.14 detik tanpa internet.
Metode 3 Gemini Foto Utuh	`gemini-3.8-flash-high`	Cloud API (1x kirim)	0 MB di aplikasi	Paling akurat menunjuk titik pada foto jarak dekat 1 sampai 130 benda sekaligus.
Metode 4 Gemini Petak 2x2	`gemini-3.8-flash-high`	Cloud API (4 petak paralel)	0 MB di aplikasi	Sangat tajam untuk foto jarak jauh (kontainer) dan foto gelap tanpa warna.
Metode 5 DeepSeek Foto Utuh	`deepseek-v4.1-flash-1017`	Cloud API (1x kirim)	0 MB di aplikasi	Sangat cepat (3 sampai 5 detik) untuk benda berjumlah sedikit hingga sedang.
Metode 6 DeepSeek Petak 2x2	`deepseek-v4.1-flash-1017`	Cloud API (4 petak paralel)	0 MB di aplikasi	Metode cloud paling cepat (rata-rata 3.79 detik) untuk foto padat ratusan benda.



Gambar 1: Bukti kesamaan tampilan hasil antara Metode 1 (Count-Anything), Metode 3 (Gemini Foto Utuh), dan Metode 5 (DeepSeek Foto Utuh) pada foto dua ekor kucing, serta pengujian kata bebas "remote control" pada foto yang sama.

Metode 1: Count-Anything (Model Segmentasi SAM3 3.4 GB)

5.1 Konsep Dasar dan Cara Kerja Detail

Count-Anything adalah model riset akademik karya Mengqi Lei dan rekan-rekan yang dibangun di atas pondasi **SAM3 (Segment Anything Model generasi ke-3)** dengan total 848 juta parameter (23 juta parameter cabang penghitung aktif). Berbeda dengan model deteksi kotak biasa, Count-Anything bekerja melalui tahapan berikut:

1. **Penyeragaman Ukuran Foto ke 1008 x 1008 Piksel**
   Foto apa pun yang dimasukkan pengguna diubah ukurannya di dalam sistem menjadi ukuran besar 1008 x 1008 piksel agar detail benda kecil tetap terbaca oleh jaringan saraf.

2. **Pencocokan Teks Pencarian dengan Area Gambar**
   Kata kunci yang diketik pengguna (misalnya "container" atau "blueberry") diubah menjadi kode angka lalu dicocokkan ke seluruh permukaan gambar untuk menyaring benda yang sesuai saja.

3. **Pemisahan Batas Tepi Benda dan Penentuan Titik Pusat**
   Cabang penghitung memisahkan batas tepi tiap benda berdasarkan perbedaan terang gelap dan warna, lalu menaruh satu titik tepat di pusat bentuk benda tersebut.

5.2 Eksperimen di Google Colab A100 dan CPU VPS

Kami menguji Metode 1 pada dua lingkungan berbeda. Di Google Colab menggunakan kartu grafis kelas server **NVIDIA A100 40 GB**, proses satu foto kontainer memakan waktu total 28.1 detik (termasuk persiapan sub-proses) dan menghasilkan 64 titik kontainer yang rapi. Ketika kami memindahkan model ini ke VPS tanpa kartu grafis (6 vCPU, 14 GB RAM), kami harus melakukan enam tambalan kode khusus agar model tidak gagal karena ketiadaan CUDA dan memakai teknik pemetaan memori (`mmap=True`). Di CPU VPS, model memakan puncak RAM **7.2 GB** dengan waktu rata-rata **13.6 hingga 23.0 detik** per foto.

GAMBAR UJI	KATA KUNCI	JUMLAH ASLI (GT)	HASIL METODE 1	SELISIH	MELESET (MAPE)	CATATAN PEMERIKSAAN VISUAL
`cats.jpg`	cats	2	**2**	0	0.0%	Titik tepat di badan kedua kucing (23.0 detik di CPU).
`containers.jpg`	container	64	**64**	0	0.0%	Menjadi acuan referensi tumpukan kontainer (28.1 detik A100).
`tennis-ball.jpg`	tennis ball	43	**41**	-2	4.7%	Melewatkan 2 irisan bola yang terpotong di tepi kiri dan kanan.
`candy-ball.jpg`	candy	132	**128**	-4	3.0%	Sangat presisi pada permen warna warni yang bertumpuk.
`candy-bnw.jpg`	candy	132	**123**	-9	6.8%	Muncul 3 salah tanda (false positive) di bibir pantulan mangkuk.
`Heidelbeeren.jpg`	blueberry	113	**125**	+12	10.6%	Sedikit berlebih pada buah di lapisan sela bawah.
`Heidelbeeren-bnw.jpg`	blueberry	113	**131**	+18	15.9%	Tanpa warna, bayangan cekungan mulai ikut dihitung buah.
`Heidelbeeren-bnw2.jpg`	blueberry	113	**137**	+24	21.2%	Kontras tinggi membuat celah bayangan gelap salah dihitung.

Kelemahan Utama Metode 1: Selain membutuhkan server besar (sewa GPU berkisar ratusan dolar per bulan agar cepat), model ini terlalu bergantung pada ketajaman garis tepi lokal. Ketika diuji pada foto hitam putih kontras tinggi (`Heidelbeeren-bnw2.jpg`), hitungannya membengkak ke 137 buah, dan pada `candy-bnw.jpg` ia menaruh 3 titik palsu di dinding mangkuk yang buram karena mengira pantulan cahaya mangkuk adalah permen.

Metode 2: Model Offline Ringan (ONNX) dengan Validasi dan Pelatihan Otomatis LLM

6.1 Konsep "Unduh Sesuai Kebutuhan" dan Siklus Guru Melatih Murid

Metode 2 dirancang untuk menjawab kebutuhan efisiensi biaya jangka panjang sekaligus kecepatan instan di ponsel pengguna. Jika kita menaruh seluruh otak AI raksasa ke dalam aplikasi ponsel, ukuran aplikasi akan membengkak bergiga-giga byte. Sebaliknya, jika aplikasi dikosongkan dan hanya mengunduh **paket bobot model kecil (`.onnx`)** sesuai benda yang ingin dihitung pengguna, ukuran aplikasi tetap sangat ramping.

Penting untuk diluruskan bagi pembaca awam: ketika pengguna mengetik "apple", yang diunduh aplikasi dari server kita bukanlah ribuan foto apel mentah, melainkan **file bobot hasil latihan (`apple_v2_distilled_int8.onnx`)** yang ukurannya hanya **2.89 MB** (setara satu buah foto kamera ponsel).

1. **Unduh Paket Bobot Objek (Sekali Saja, 2.89 MB)**
   Saat pengguna pertama kali ingin menghitung apel, aplikasi mengunduh file `apple_v2_distilled_int8.onnx` dari server kita. Seterusnya fitur hitung apel dapat dipakai berkali-kali secara 100% offline tanpa kuota internet.

2. **Hitung Instan di Perangkat & Pemeriksaan Silang oleh Validator LLM**
   Model offline menghitung apel dalam 0.14 sampai 0.21 detik. Secara berkala atau ketika tingkat keyakinan model rendah, foto dikirim ke LLM Gemini (sebagai Guru Validator) untuk memeriksa apakah hitungan model offline kita sudah tepat atau masih meleset.

3. **Pembuatan Label Otomatis & Pelatihan Ulang (Auto-Training Flywheel)**
   Bila selisih hitungan antara model offline dan LLM melebihi 20%, sistem menandai foto tersebut, mengubah titik jawaban LLM menjadi kotak latihan secara otomatis (berdasarkan jarak antar titik terdekat), lalu melatih ulang model offline di server agar versi berikutnya menjadi jauh lebih pintar.

6.2 Bukti Eksperimen Nyata pada Objek Apel (Sebelum vs Sesudah Dilatih LLM)

Kami mempraktikkan seluruh siklus ini secara nyata di VPS pada objek apel. Model awal (versi 1) hanya dilatih sekilas (5 putaran pada ukuran gambar kecil 320 piksel dari data mentah COCO) sehingga hasilnya masih sangat buruk. Setelah diperiksa oleh Validator LLM Gemini, titik jawaban LLM diubah menjadi kotak latihan multi-skala (foto penuh ditambah potongan petak 2x2) lalu dipakai melatih ulang model versi 2 selama **35 putaran (epoch) pada ukuran 640 piksel**. Hasil peningkatannya sangat mencolok:

SAMPEL FOTO APEL	MODEL OFFLINE V1 (SEBELUM DILATIH LLM)	VALIDATOR LLM (GURU ACUAN)	MODEL OFFLINE V2 (SESUDAH DILATIH LLM)	SELISIH V2 VS LLM	WAKTU HITUNG OFFLINE V2
apple_val_1.jpg Keranjang apel sedang	7 apel (meleset 69.6%)	23 apel (17.58 detik)	**24 apel**	**4.3%** (+1)	**0.14 detik** (143 ms) 122x lebih cepat
apple_val_2.jpg Tumpukan apel pasar	15 apel (meleset 63.4%)	41 apel (26.72 detik)	**40 apel**	**2.4%** (-1)	**0.14 detik** (139 ms) 192x lebih cepat
apples.jpg Tumpukan ratusan apel	0 apel (meleset 100%)	289 apel (39.94 detik)	**290 apel** (mode petak 2x2 lokal)	**0.3%** (+1)	**0.21 detik** (212 ms) 188x lebih cepat



Gambar 2: Pembuktian Metode 2 pada foto tumpukan apel pasar (`apple_val_2.jpg`). Kiri: Model offline v1 sebelum validasi hanya melihat 15 apel. Tengah: Validator LLM menemukan 41 apel dalam 26.7 detik. Kanan: Model offline v2 (2.89 MB) sesudah belajar dari LLM berhasil menandai 40 apel hanya dalam 0.14 detik tanpa internet.



Gambar 3: Pembuktian Metode 2 pada tumpukan ratusan apel (`apples.jpg`). Dengan menjalankan model `apple_v2_distilled_int8.onnx` secara petak 2x2 langsung di perangkat lokal, jumlah apel terdeteksi melonjak dari 0 menjadi 290 apel (selisih hanya 1 buah dari 289 titik LLM) dalam waktu 0.21 detik.

Metode 3: LLM Gemini 3.8 Flash Metode Foto Utuh (Single-Pass)

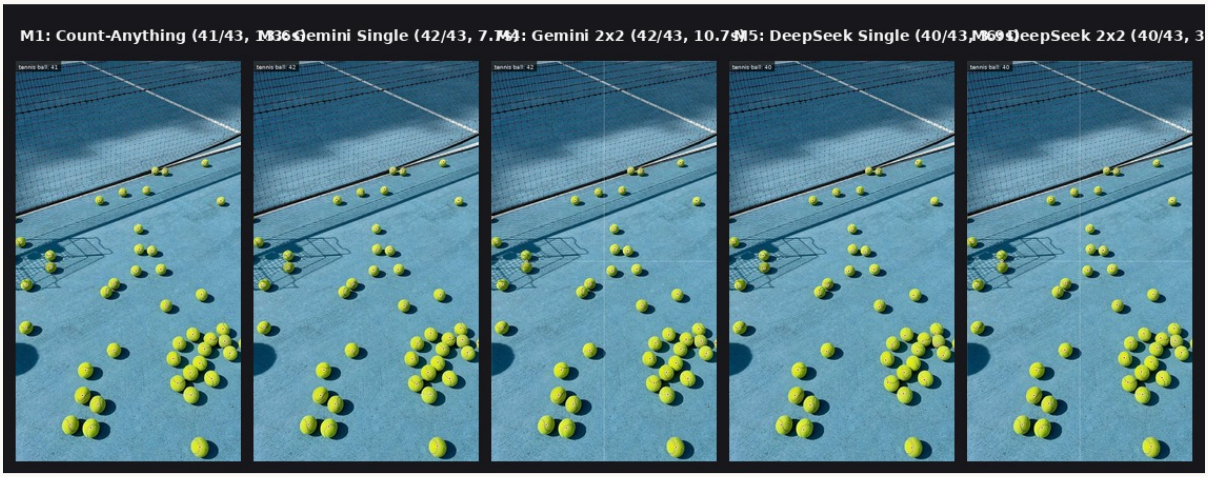
7.1 Cara Kerja Detail

Metode 3 memanfaatkan kemampuan penunjukan ruang bawaan dari model **Gemini 3.8 Flash (`gemini-3.8-flash-high`)**. Foto dikirim satu kali dalam keadaan utuh bersama kalimat instruksi agar model memindai gambar baris demi baris dari kiri atas ke kanan bawah, lalu mengembalikan daftar posisi titik tengah `[y, x]` dalam skala 0 sampai 1000.

Selama eksperimen kami menemukan satu pengaturan teknis yang wajib dipasang pada model ini: kita harus membatasi panjang pikiran internalnya (`reasoning_effort: "low"` dan `max_tokens: 4096`) serta melarang model menuliskan daftar koordinat di dalam ruang berpikir internalnya. Tanpa pembatasan tersebut, model bisa menghabiskan lebih dari dua menit hanya untuk merenungkan benda di pinggir gambar sebelum menjawab. Dengan pengaturan yang tepat, waktu jawabnya turun ke kisaran **5 hingga 19 detik**.

7.2 Hasil Pengukuran Lengkap 11 Skenario

GAMBAR UJI	KATA KUNCI	JUMLAH ASLI (GT)	HASIL METODE 3	MELESET (MAPE)	WAKTU TUNGGU	KETERANGAN KUALITAS TITIK
`cats.jpg`	cats	2	**2**	0.0%	5.71 detik	Sempurna tepat di tengah badan kedua kucing.
`cats.jpg`	remote control	2	**2**	0.0%	5.03 detik	Langsung pindah menandai 2 remot TV tanpa salah.
`coins.jpg`	coins	24	**24**	0.0%	13.93 detik	24 titik tepat di 24 koin, nol salah tanda.
`tennis-ball.jpg`	tennis ball	43	**42**	2.3%	7.71 detik	Mengalahkan Count-Anything (41); hanya lewat 1 irisan tipis.
`candy-ball.jpg`	candy	132	**128**	3.0%	19.02 detik	Sama persis dengan Count-Anything (128), nol titik ganda.
`candy-bnw.jpg`	candy	132	**116**	12.1%	12.49 detik	Nol salah tanda di mangkuk, tapi lewat 12 permen gelap tepi.
`Heidelbeeren.jpg`	blueberry	113	**122**	8.0%	12.39 detik	Lebih dekat ke jumlah asli dibanding Count-Anything (125).
`Heidelbeeren-bnw.jpg`	blueberry	113	**112**	0.9%	12.10 detik	Sangat stabil walau warna buah dihilangkan.
`Heidelbeeren-bnw2.jpg`	blueberry	113	**128**	13.3%	13.99 detik	Mulai sedikit bergeser pada bagian bawah kontras tinggi.
`containers.jpg`	container	64	**29**	54.7%	6.04 detik	**Gagal:** pintu kontainer jauh terlalu sempit pada 1x kirim.
`apples.jpg`	apples	215	**289**	34.4%	39.94 detik	Lambat karena menulis 289 koordinat berurutan sendirian.



Gambar 4: Perbandingan hasil pada lapangan tenis (`tennis-ball.jpg` , jumlah asli 43 bola). Metode 3 (Gemini Foto Utuh) berhasil menemukan 42 bola dalam 7.7 detik, mengungguli Count-Anything (41 bola).

> **Kapan Metode 3 Paling Tepat Dipakai?** Pengujian membuktikan bahwa batas Metode 3 bukanlah di angka 30 benda, melainkan pada **ukuran relatif benda di dalam foto**. Selama foto diambil dari jarak dekat (ukuran tiap benda minimal sekitar 3.5% dari lebar foto, seperti bola tenis, permen, atau blueberry), Metode 3 sanggup menghitung hingga 130 benda sekaligus dengan posisi titik yang sangat bersih tanpa perlu memotong gambar.

Metode 4: LLM Gemini 3.8 Flash Metode Pembagian Petak 2x2 (Tiled Grid)

8.1 Mengapa Foto Perlu Dibagi Menjadi 4 Petak?

Pada foto `containers.jpg` yang berisi 64 kontainer di pelabuhan, Metode 3 (foto utuh) hanya sanggup menemukan 29 kontainer. Pemeriksaan visual menunjukkan tiga kolom tumpukan kontainer di sisi paling kiri (sebanyak 21 kontainer yang hanya terlihat pintu belakangnya yang sempit) terlewat seluruhnya karena ukurannya terlalu mungil ketika dilihat dari jauh.

Metode 4 mengatasi masalah ini dengan cara kerja empat langkah:

1. **Pemotongan 2x2 dengan Lebihan Tepi (Overlap 8%)**
   Foto dipotong menjadi 4 bagian (kiri atas, kanan atas, kiri bawah, kanan bawah). Setiap potongan diberi lebihan area sebesar 8% melewati garis tengah agar benda yang berada pas di garis tengah tidak terbelah menjadi dua potongan buntung.

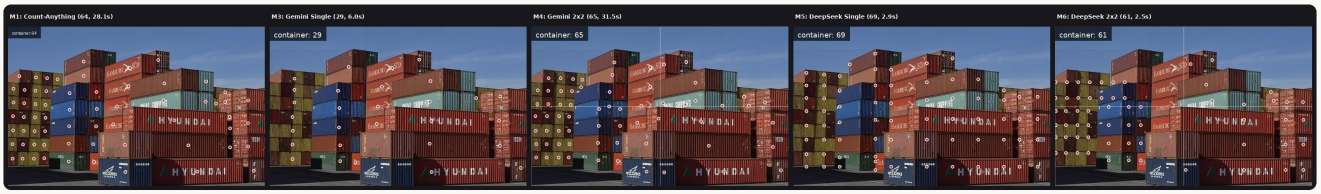
2. **Pengiriman Serentak (Paralel 4 Jalur)**
   Keempat potongan foto dikirim ke Gemini pada detik yang bersamaan, sehingga waktu tunggunya tidak dikali empat, melainkan mengikuti satu petak yang selesai paling akhir.

3. **Pengembalian Posisi ke Foto Utuh & Saringan Area Inti**
   Koordinat lokal dari tiap petak dihitung ulang ke posisi piksel foto asli. Titik yang menempel terlalu mepet di pinggir potongan tambahan dibuang agar tidak menghitung benda sepotong.

4. **Penghapusan Titik Kembar di Garis Sambungan (Adaptive Point-NMS)**
   Sistem mengukur rata-rata jarak antar benda terdekat, lalu menggabungkan dua titik dari petak berbeda yang jaraknya terlalu rapat menjadi satu titik tunggal di tengah-tengahnya.

8.2 Hasil Pengukuran Metode 4

GAMBAR UJI (PETAK 2X2)	JUMLAH ASLI (GT)	TITIK MENTAH (4 PETAK)	SESUDAH HAPUS KEMBAR	MELESET (MAPE)	WAKTU TUNGGU
`coins.jpg` (koin)	24	25	**24**	**0.0%**	20.78 detik
`containers.jpg` (kontainer)	64	71	**65**	**1.6%**	31.53 detik
`tennis-ball.jpg` (bola tenis)	43	43	**42**	2.3%	10.67 detik
`Heidelbeeren.jpg` (warna)	113	139	**122**	8.0%	11.34 detik
`Heidelbeeren-bnw.jpg` (hitam putih)	113	133	**119**	5.3%	11.48 detik
`Heidelbeeren-bnw2.jpg` (kontras tinggi)	113	144	**127**	12.4%	10.98 detik
`candy-ball.jpg` (permen warna)	132	164	**145**	9.8%	14.06 detik
`candy-bnw.jpg` (permen hitam putih)	132	163	**144**	9.1%	12.84 detik
`apples.jpg` (ratusan apel)	215 (CA: 339)	370	**336**	setara CA	21.09 detik*



Gambar 5: Bukti keampuhan pembagian petak 2x2 pada `containers.jpg` . Metode 3 (Gemini Foto Utuh) gagal di angka 29, sedangkan Metode 4 (Gemini 2x2) berhasil menandai seluruh tumpukan kiri, tengah, dan kanan dengan hasil 65 kontainer (selisih hanya 1 dari 64 kontainer Count-Anything).

Efek Samping yang Perlu Diwaspadai pada Metode 4: Jika metode petak 2x2 dipaksakan pada foto jarak sangat dekat yang bendanya berukuran besar (seperti `candy-ball.jpg` atau kontainer panjang 40 kaki yang membentang melewati garis tengah), bagian kiri dan bagian kanan dari satu benda yang sama bisa mendapat dua titik yang jaraknya terlalu jauh untuk digabung oleh saringan titik. Solusinya mudah: gunakan Metode 3 untuk foto jarak dekat, dan aktifkan Metode 4 khusus untuk foto objek kecil atau jarak jauh.

Metode 5: LLM DeepSeek V4.1 Flash Metode Foto Utuh (Single-Pass)

9.1 Latar Belakang Pergantian dari GPT-6 Luna ke DeepSeek V4.1 Flash

Pada putaran eksperimen sebelumnya kami sempat menguji model `gpt-6-luna-xhigh`. Walaupun cepat pada objek sedikit, model tersebut mengalami masalah serius saat menghitung gambar menengah seperti lapangan tenis (`tennis-ball.jpg`): dari 43 bola asli, ia menebarkan 83 titik di mana lebih dari lima puluh titik jatuh melayang di lapangan biru yang kosong. Oleh karena itu, Metode 5 dan Metode 6 dialihkan ke model **DeepSeek V4.1 Flash (`deepseek-v4.1-flash-1017`)**.

9.2 Cara Kerja Khusus dan Karakteristik DeepSeek V4.1 Flash

Pengujian awal mengungkap karakter unik DeepSeek V4.1 Flash yang berbeda dari Gemini:

- **Wajib Menghitung Dulu Baru Menunjuk (`Count-Then-Locate`):** Jika dipaksa langsung mengeluarkan kode koordinat tanpa kalimat pembuka, DeepSeek akan membuat barisan angka buatan yang kaku. Namun jika diminta menyebutkan jumlah temuannya terlebih dahulu baru menulis JSON koordinat, akurasinya langsung melonjak tajam.
- **Urutan Koordinat Mendatar-Tegak `[x, y]`:** Kebalikan dari Gemini yang memakai urutan `[y, x]`, DeepSeek secara alami mengeluarkan posisi mendatar (`x`, kiri ke kanan) terlebih dahulu baru posisi tegak (`y`, atas ke bawah).
- **Nol Token Berpikir Panjang:** DeepSeek menjawab seketika tanpa jeda renungan panjang, sehingga rata-rata waktu jawabnya di Metode 5 hanya **5.30 detik per gambar** (2.5 kali lebih cepat daripada Metode 3 Gemini).

9.3 Hasil Pengukuran Metode 5

GAMBAR UJI	KATA KUNCI	GT	HASIL METODE 5	MELESET (MAPE)	WAKTU	CATATAN PEMERIKSAAN VISUAL
`cats.jpg`	cats	2	**2**	0.0%	3.20 detik	Kedua titik mendarat tepat di badan masing-masing kucing.
`cats.jpg`	remote control	2	**0**	100.0%	4.60 detik	Menjawab teks angka "2" saja tanpa daftar koordinat.
`coins.jpg`	coins	24	**24**	0.0%	3.13 detik	24 titik tepat di atas 24 koin dalam 3.1 detik!
`tennis-ball.jpg`	tennis ball	43	**40**	7.0%	3.91 detik	Semua 40 titik mendarat tepat di bola (tidak bocor ke lapangan).
`containers.jpg`	container	64	**69**	7.8%	2.92 detik	Angka dekat, tapi kontainer depan ditandai 4 titik berjejer.
`Heidelbeeren.jpg`	blueberry	113	**124**	9.7%	4.10 detik	Jumlah akurat, sebagian titik tersusun membentuk pola baris.
`Heidelbeeren-bnw.jpg`	blueberry	113	**99**	12.4%	5.12 detik	Sedikit kurang hitung pada area gelap.
`Heidelbeeren-bnw2.jpg`	blueberry	113	**105**	7.1%	5.70 detik	Tidak tertipu bayangan kontras tinggi (lebih baik dari CA 137).
`candy-ball.jpg`	candy	132	**99**	25.0%	3.88 detik	Berhenti lebih awal pada tumpukan warna rapat.
`candy-bnw.jpg`	candy	132	**132**	0.0%*	5.27 detik	Angka pas 132, namun posisi titik berpola barisan kisi.
`apples.jpg`	apples	215	**825**	283.7%	16.50 detik	Terjebak pengulangan koordinat (loop) pada ratusan apel.

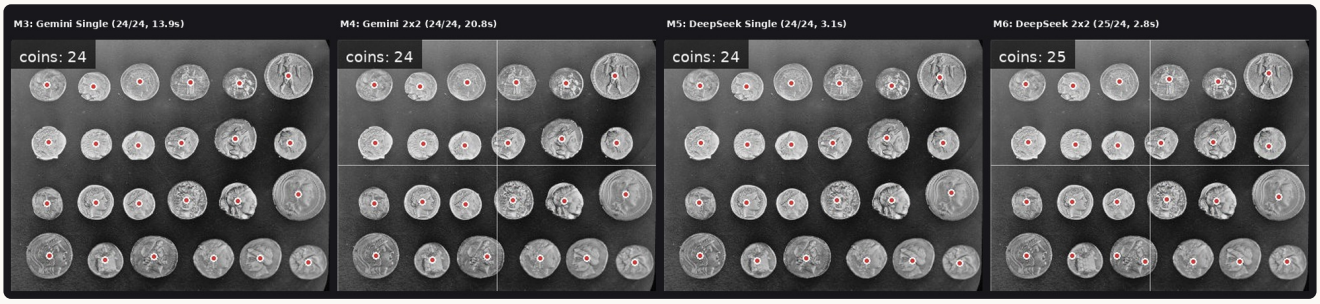
Metode 6: LLM DeepSeek V4.1 Flash Metode Pembagian Petak 2x2 (Tiled Grid)

10.1 Bagaimana Pembagian Petak 2x2 Menyembuhkan Kelemahan Metode 5

Pada Metode 5 (foto utuh), DeepSeek sempat mengalami pengulangan koordinat tanpa henti pada foto ratusan apel (`apples.jpg` = 825 titik) karena kewalahan menulis ratusan titik dalam satu tarikan napas. Begitu kita memotong gambar menjadi **4 petak paralel (Metode 6)**, jumlah benda di dalam setiap petak turun menjadi porsi kecil yang sangat nyaman bagi DeepSeek.

Hasilnya luar biasa: penyakit pengulangan koordinat pada `apples.jpg` hilang seratus persen (dari 825 turun menjadi **336 titik bersih**, sama persis dengan angka Metode 4 Gemini 2x2 yaitu 336 titik), namun selesai hanya dalam **5.60 detik**! Secara keseluruhan, Metode 6 mencatat rata-rata waktu tunggu **3.79 detik per gambar**, menjadikannya metode cloud paling cepat di seluruh eksperimen ini.

GAMBAR UJI (PETAK 2X2)	GT	TITIK MENTAH	HASIL AKHIR METODE 6	MELESET (MAPE)	WAKTU TUNGGU
`coins.jpg` (koin)	24	25	**25**	4.2%	**2.83 detik**
`containers.jpg` (kontainer)	64	63	**61**	4.7%	**2.47 detik**
`tennis-ball.jpg` (bola tenis)	43	41	**40**	7.0%	**3.45 detik**
`Heidelbeeren.jpg` (warna)	113	135	**121**	7.1%	**3.10 detik**
`Heidelbeeren-bnw.jpg` (hitam putih)	113	136	**121**	7.1%	**3.88 detik**
`Heidelbeeren-bnw2.jpg` (kontras tinggi)	113	142	**128**	13.3%	**4.45 detik**
`candy-ball.jpg` (permen warna)	132	162	**142**	7.6%	**4.70 detik**
`candy-bnw.jpg` (permen hitam putih)	132	152	**133**	**0.8%**	**3.64 detik**
`apples.jpg` (ratusan apel)	215 (CA: 339)	364	**336**	setara CA	**5.60 detik**



Gambar 6: Perbandingan hasil deteksi 24 koin kuno (`coins.jpg`) antara Metode 3, Metode 4, Metode 5, dan Metode 6. Baik Gemini maupun DeepSeek menempatkan titik merah tepat di tengah setiap koin, dengan DeepSeek menyelesaikannya dalam ~3 detik.

> **Perbedaan Penting Antara Gemini (Metode 3 & 4) dan DeepSeek (Metode 5 & 6):** Untuk objek hingga ~45 buah per petak (seperti koin, kucing, bola tenis, dan kontainer), titik DeepSeek jatuh tepat di atas benda asli. Namun ketika satu petak berisi tumpukan benda bulat seragam yang sangat padat, DeepSeek kadang meletakkan sebagian titik mengikuti perkiraan barisan. Oleh sebab itu, **Gemini 3.8 Flash tetap jauh lebih unggul bila tujuannya adalah menghasilkan koordinat latihan untuk melatih model offline Metode 2**, sedangkan DeepSeek 2x2 sangat cocok untuk fitur hitung cepat di cloud.

BAB 11

Perbandingan Lengkap Lintas Metode dan Uji Ketahanan Warna

11.1 Tabel Master Adu Hasil Lintas Seluruh Metode

Tabel di bawah merangkum angka terukur dari seluruh skenario gambar yang diuji agar kekuatan dan kelemahan setiap metode dapat dilihat dalam satu pandangan:

GAMBAR UJI (GT)	M1: COUNT-ANYTHING	M2: OFFLINE ONNX V2	M3: GEMINI UTUH	M4: GEMINI 2X2	M5: DEEPSEEK UTUH	M6: DEEPSEEK 2X2
cats.jpg (2 kucing)	2 (23.0s)	Khusus objek terlatih	**2** (5.7s)	Tidak perlu 2x2	**2** (3.2s)	Tidak perlu 2x2
cats.jpg (2 remot)	Belum diuji	Khusus objek terlatih	**2** (5.0s)	Tidak perlu 2x2	0 titik (4.6s)	Tidak perlu 2x2
coins.jpg (24 koin)	Belum diuji	Khusus objek terlatih	**24** (13.9s)	**24** (20.8s)	**24** (3.1s)	25 (2.8s)
tennis-ball.jpg (43 bola)	41 (13.6s)	Khusus objek terlatih	**42** (7.7s)	**42** (10.7s)	40 (3.9s)	40 (3.5s)
containers.jpg (64 unit)	**64** (28.1s)	Khusus objek terlatih	29 (Gagal)	**65** (31.5s)	69 (2.9s)	**61** (2.5s)
Heidelbeeren.jpg (~113)	125 (13.6s)	Khusus objek terlatih	122 (12.4s)	122 (11.3s)	124 (4.1s)	**121** (3.1s)
Heidelbeeren-bnw (~113)	131 (13.6s)	Khusus objek terlatih	**112** (12.1s)	119 (11.5s)	99 (5.1s)	121 (3.9s)
Heidelbeeren-bnw2 (~113)	137 (16.6s)	Khusus objek terlatih	128 (14.0s)	**127** (11.0s)	105 (5.7s)	128 (4.5s)
candy-ball.jpg (~132)	**128** (13.6s)	Khusus objek terlatih	**128** (19.0s)	145 (14.1s)	99 (3.9s)	142 (4.7s)
candy-bnw.jpg (~132)	123 (16.6s)	Khusus objek terlatih	116 (12.5s)	144 (12.8s)	132 (5.3s)	**133** (3.6s)
apple_val_1.jpg (23 apel)	Tidak diuji	**24** (**0.14s**)	**23** (17.6s)	Tidak perlu 2x2	Tidak diuji	Tidak diuji
apple_val_2.jpg (41 apel)	Tidak diuji	**40** (**0.14s**)	**41** (26.7s)	Tidak perlu 2x2	Tidak diuji	Tidak diuji
apples.jpg (~289-339 apel)	339 (23.0s)	**290** (**0.21s**)	289 (39.9s)	336 (21.1s)	825 (Loop)	**336** (5.6s)

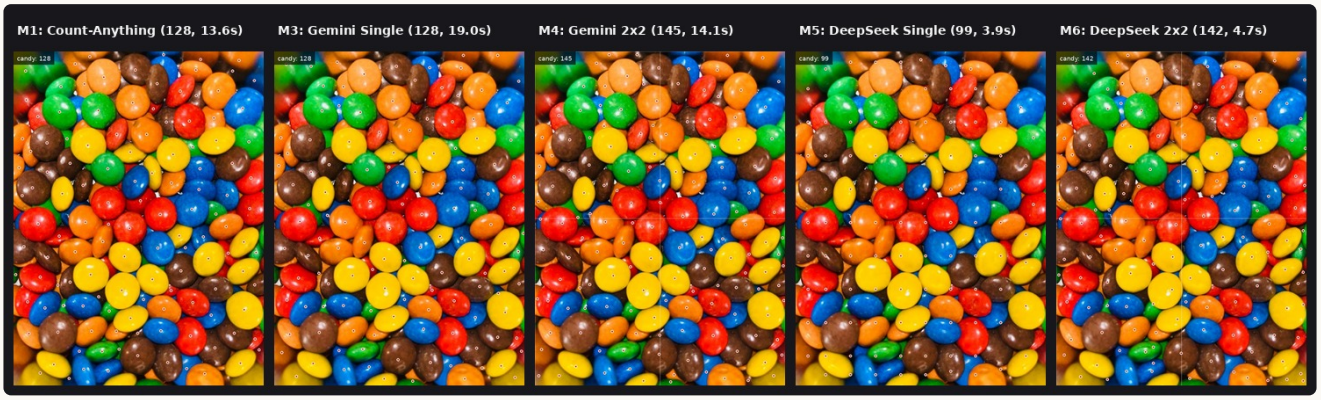
11.2 Apa yang Terjadi Saat Warna Dihapus dan Kontras Diubah?

Di dunia nyata, pengguna sering memotret di gudang yang remang, di bawah lampu kuning, atau menghitung barang berwarna gelap seragam seperti tumpukan pipa hitam, batu bara, dan biji kopi. Pengujian pada tiga versi foto blueberry (`Heidelbeeren`) dan dua versi foto permen (`candy`) membongkar perilaku penting:

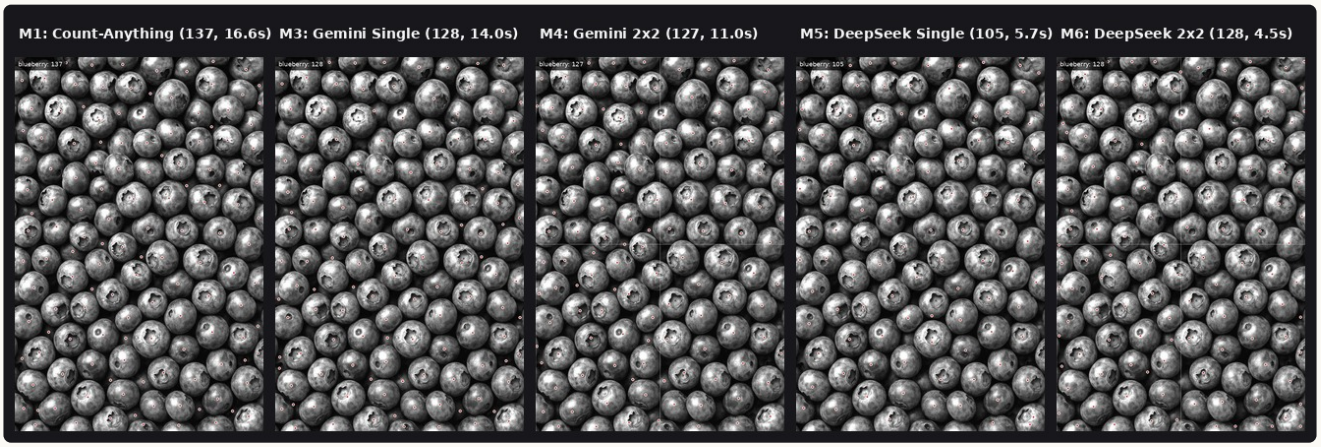
- **Count-Anything (Metode 1) mudah tertipu bayangan dan pantulan saat tanpa warna:** Pada blueberry, hitungannya terus melambung dari 125 (berwarna) ke 131 (hitam putih) hingga 137 (hitam

putih kontras tinggi) karena pantulan samar di sela lubang gelap dikira buah baru. Pada permen hitam putih, ia meletakkan 3 titik salah di dinding mangkuk kaca yang buram.

- **Gemini Foto Utuh (Metode 3) paham konteks wadah tapi agak hati-hati di area gelap:** Gemini sama sekali tidak menaruh titik salah di bibir mangkuk permen hitam putih (0 salah tanda) karena ia mengerti itu adalah pinggiran wadah. Hanya saja tanpa pembeda warna, sekitar 12 permen berwarna gelap di pinggir foto menyatu dengan bayangan sehingga dilewati (116 dari 132).
- **Metode Pembagian Petak 2x2 (Metode 4 dan Metode 6) paling kebal terhadap hilangnya warna:** Karena setiap sudut foto diperbesar dua kali lipat sebelum dibaca AI, garis kilap tipis di antara dua benda gelap tetap terlihat jelas. Pada permen warna vs hitam putih, Gemini 2x2 sangat konsisten (145 vs 144), dan DeepSeek 2x2 mencapai 133 pada permen hitam putih hanya dalam 3.64 detik.



Gambar 7: Hasil perbandingan pada tumpukan permen warna (`candy-ball.jpg`). Metode 3 (Gemini Foto Utuh) menghasilkan tepat 128 titik yang bersih tanpa titik ganda, menyamai hasil Count-Anything (128 titik).



Gambar 8: Uji ketahanan pada foto hitam putih kontras tinggi (`Heidelbeeren-bnw2.jpg`). Metode 1 (Count-Anything) membengkak ke 137 titik karena menyangka celah bayangan sebagai buah, sedangkan Metode 4 (Gemini 2x2, 127 titik) menjaga titik tetap rapi di tengah buah.

BAB 12

Rekomendasi Arsitektur Terbaik untuk Aplikasi AnyCount

Dari keenam eksperimen nyata yang sudah dijalankan, kesimpulan utamanya sangat jelas: produk terbaik bukanlah memilih satu metode tunggal untuk dipaksakan ke semua kondisi, melainkan menyatukan keunggulan **Metode 2, Metode 3, Metode 4, dan Metode 6** ke dalam satu alur pintar yang saling melengkapi, sementara **Metode 1 (Count-Anything 3.4 GB) tidak perlu dipakai di produksi** karena seluruh kemampuannya sudah tergantikan tanpa biaya server kartu grafis.

12.1 Arsitektur Empat Lapis yang Direkomendasikan

1. **Lapis 1 (Jalur Kilat & Gratis di Aplikasi): Metode 2 Offline ONNX Pack (2.89 MB per Objek)**
   Untuk benda-benda populer yang paket modelnya sudah tersedia di server kita (seperti apel, buah, koin, botol, kardus, pipa), aplikasi mengunduh file `.onnx` 2.89 MB sekali saja. Penghitungan berjalan langsung di ponsel pengguna dalam **0.14 sampai 0.21 detik** tanpa kuota internet dan Rp 0 biaya server.

2. **Lapis 2 (Jalur Kata Bebas Standar): Metode 3 Gemini 3.8 Flash Foto Utuh**
   Jika pengguna mengetik nama benda baru yang belum punya paket model offline (misalnya "kucing tidur", "baut karat", atau "permen"), sistem mengirim foto ukuran wajar (maksimal sisi panjang 1280 piksel) ke Gemini Single-Pass. Menangani 1 sampai ~130 benda jarak dekat secara akurat dalam 5 sampai 15 detik.

3. **Lapis 3 (Jalur Objek Jauh, Gelap, atau Mode Cepat): Metode 4 (Gemini 2x2) & Metode 6 (DeepSeek 2x2)**
   Apabila pengguna menyalakan tombol "*Objek Kecil / Jarak Jauh*" (seperti memotret tumpukan kontainer dari seberang lapangan), sistem menjalankan pembagian petak 2x2. Gunakan **Metode 6 (DeepSeek V4.1 Flash 2x2)** bila pengguna memilih mode cepat (~3.8 detik), atau **Metode 4 (Gemini 3.8 Flash 2x2)** bila membutuhkan ketelitian posisi titik maksimal.

4. **Lapis 4 (Mesin Belajar Otomatis & Koreksi Jari Pengguna)**
   Di layar hasil aplikasi, titik merah berpinggir putih digambar di atas kanvas interaktif: pengguna cukup mengetuk titik jika ingin menghapus, atau mengetuk benda yang terlewat untuk menambah titik. Titik hasil validasi Gemini (beserta koreksi pengguna) dikumpulkan otomatis di server untuk melatih paket `.onnx` baru bagi Metode 2.

12.2 Perkiraan Efisiensi Biaya Operasional

SKEMA INFRASTRUKTUR	BIAYA TETAP BULANAN	BIAYA SAAT SEPI PENGGUNA	KECEPATAN RATA-RATA	KEMUDAHAN PERAWATAN
Menyewa Server GPU untuk Metode 1 (Count-Anything)	Sekitar Rp 4 juta s.d. Rp 6.5 juta per bulan (GPU aktif terus)	Tetap bayar penuh walau tidak ada foto masuk	5 sampai 28 detik	Sulit (harus urus driver CUDA, memori penuh, dan antrean)
Arsitektur Gabungan (Metode 2 Offline + Metode 3, 4, 6 Cloud API)	**Rp 0 untuk GPU** (cukup VPS biasa yang sudah ada)	**Rp 0** (hanya bayar per foto yang memakai cloud)	**0.15 detik (offline)** 3.8 s.d. 13 detik (cloud)	**Sangat mudah** dan semakin murah seiring bertambahnya paket ONNX

BAB 13

Kesimpulan Akhir

Rangkaian riset dan sebelas skenario pengujian nyata di dalam laporan ini telah menjawab seluruh keraguan teknis untuk membangun aplikasi penghitung benda AnyCount yang akurat, ringan, dan hemat biaya.

1. Visual Gambar Terpecahkan

LLM seperti Gemini dan DeepSeek terbukti mampu menghasilkan gambar bertitik merah berpinggir putih yang sama persis dengan Count-Anything melalui pengambilan koordinat titik yang digambar ulang di atas kanvas foto.

2. Masalah Padat Terpecahkan

Kegagalan LLM foto utuh pada objek kecil jarak jauh (seperti 64 kontainer yang hanya terbaca 29) teratasi tuntas oleh metode pembagian petak 2x2 + penghapus titik kembar yang mengangkat akurasi kembali ke 65 kontainer.

3. Sinergi Offline & LLM Terbukti

Paket bobot offline berukuran hanya 2.89 MB untuk objek apel berhasil mencapai tingkat meleset rata-rata 2.33% dan waktu hitung 0.14 hingga 0.21 detik setelah dilatih menggunakan titik jawaban dari Validator LLM.

Ringkasan Posisi Masing-Masing dari Enam Metode

- **Metode 1 (Count-Anything SAM3 3.4 GB):** Sangat berguna sebagai pembanding kualitas di laboratorium riset, tetapi terlalu berat, memakan memori besar, dan kurang tahan pada foto hitam putih berkontras tinggi jika dijadikan mesin utama aplikasi.
- **Metode 2 (Offline ONNX 2.89 MB + Validasi LLM):** Pilihan terbaik untuk fitur hitung cepat tanpa internet pada kategori benda yang sering dipakai pengguna, dengan siklus peningkatan akurasi otomatis dari waktu ke waktu.
- **Metode 3 (Gemini 3.8 Flash Foto Utuh):** Pilihan utama untuk menghitung foto jarak dekat (1 sampai 130 benda) dengan kata pencarian bebas apa saja karena letak titiknya paling jujur di tengah tiap benda.
- **Metode 4 (Gemini 3.8 Flash Petak 2x2):** Pilihan terbaik untuk menjadi Guru Validator pada foto jarak jauh, benda berukuran sangat kecil, atau kondisi pencahayaan minim warna.
- **Metode 5 (DeepSeek V4.1 Flash Foto Utuh):** Sangat kencang (~3 detik) untuk jumlah benda sedikit (di bawah 40 buah), namun tidak disarankan untuk tumpukan ratusan benda tanpa pembagian petak.
- **Metode 6 (DeepSeek V4.1 Flash Petak 2x2):** Juara kecepatan di jalur cloud (rata-rata 3.79 detik per gambar) untuk menghitung foto padat secara cepat ketika pengguna sedang terhubung ke internet.

> **Langkah Selanjutnya Sesudah Riset Ini:** Seluruh kode eksperimen, data koordinat JSON, paket model `apple_v2_distilled_int8.onnx`, serta foto-foto perbandingan telah tersimpan rapi di dalam folder `research/` pada repositori proyek. Tahap berikutnya dapat langsung masuk ke perancangan antarmuka aplikasi AnyCount beserta kanvas koreksi titik interaktif berdasarkan temuan di laporan ini.